Transform Your Future with
"HELLO WORLD!"
INSTITUTE

Why learn with us ?

Training From Experts

One - on – One Guidance

100% Practical

Real-Time Projects & Case Studies

Placement Preparation Support

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Contact :- 8085896740 @ hello_world_institute Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Dictionary

- **Dictionary** is a collection of keys values, used to store data values like a map, which, unlike other data types which hold only a single value as an element.
- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

NOTE : As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.

Dictionaries are written with curly brackets, and have keys and values:

Example :

```
Dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(Dict1)
```

Dictionary Items

- Dictionary items are ordered, changeable, and does not allow duplicates.
- Dictionary items are presented in key:value pairs, and can be referred to by using the key name. **Example**

```
dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(dict1["brand"])
```

Ordered or Unordered?

-

- As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.
- When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.
- Unordered means that the items does not have a defined order, you cannot refer to an item by using an index.

Changeable

Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed

Dictionaries cannot have two items with the same key:

Example

```
dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(dict1)
```

Dictionary Length

To determine how many items a dictionary has, use the len() function:

Example

```
print(len(dict1))
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

Example : String, int, boolean, and list data types:

```
dict1 = {
  "brand": "Ford",
  "electric": False,
  "year": 1964,
  "colors": ["red", "white", "blue"]
}
```

type()

From Python's perspective, dictionaries are defined as objects with the data type 'dict':

Example :

<class 'dict'>

Print the data type of a dictionary:

```
dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(type(dict1))
```

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets:

Example :Get the value of the "model" key:

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

```
dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = dict1["model"]
```

There is also a method called get() that will give you the same result:

Example :Get the value of the "model" key:

```
x = dict1.get("model")
```

Get Keys :

The keys() method will return a list of all the keys in the dictionary.

Example :Get a list of the keys:

```
x = dict1.keys()
```

The list of the keys is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.

Example :

Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.keys()
print(x)#beforethechangecar["color"]="white"
print(x) #after the change
```

Get Values :

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

The values() method will return a list of all the values in the dictionary.

Example : Get a list of the values:

```
x = dict1.values()
```

The list of the values is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list.

Example : Make a change in the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.values()
print(x) #before the change car["year"] = 2020
print(x) #after the change
```

Get Items

The items() method will return each item in a dictionary, as tuples in a list.

Example : Get a list of the key:value pairs

```
x = dict1.items()
```

Example :

The returned list is a *view* of the items of the dictionary, meaning that any changes done to the dictionary will be reflected in the items list.

Add a new item to the original dictionary, and see that the items list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.items()
print(x) #before the change car["color"] = "red"
print(x) #after the change
```

Check if Key Exists

To determine if a specified key is present in a dictionary use the in keyword:

Example:

Check if "model" is present in the dictionary:

```
dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
if "model" in dict1:
  print("Yes, 'model' is one of the keys in the dict1 dictionary")
```

Change Values :

You can change the value of a specific item by referring to its key name:

Example : Change the "year" to 2018:

```
dict1 = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
dict1["year"] = 2018
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Update Dictionary

- The update() method will update the dictionary with the items from the given argument.
- The argument must be a dictionary, or an iterable object with key:value pairs.

Example : Update the "year" of the car by using the update() method:

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
dict1.update({"year": 2020})
```

Adding Items

Adding an item to the dictionary is done by using a new index key and assigning a value to it:

Example

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
dict1["color"] = "red"

print(dict1)
```

Removing Items

There are several methods to remove items from a dictionary:

Example : The pop() method removes the item with the specified key name:

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
dict1.pop("model")
print(dict1)
```

The popitem() method removes the last inserted item (in versions before 3.7, a random item is removed instead):

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
dict1.popitem()
print(dict1)
```

Example : The del keyword removes the item with the specified key name:

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del dict1["model"]
print(dict1)
```

Example : The del keyword can also delete the dictionary completely:

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del dict1
print(thisdict) #this will cause an error because "dict1" no longer exists.
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Example : The clear() method empties the dictionary:

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
dict1.clear()

 print(dict1)
```

Loop Through a Dictionary

- You can loop through a dictionary by using a for loop.
- When looping through a dictionary, the return values are the *keys* of the dictionary, but there are methods to return the *values* as well.

Example : Print all key names in the dictionary, one by one:

```
for x in thisdict:
 print(x)
```

Example : Print all *values* in the dictionary, one by one:

```
for x in thisdict:
print(thisdict[x])
```

Example : You can also use the values() method to return values of a dictionary:

```
for x in thisdict.values():
print(x)
```

Example : You can use the keys() method to return the keys of a dictionary:

```
for x in thisdict.keys():
 print(x)
```

Example: Loop through both *keys* and *values*, by using the items() method:

```
for x, y in thisdict.items():
print(x, y)
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Copy a Dictionary

- You cannot copy a dictionary simply by typing dict2 = dict1, because: dict2 will only be a *reference* to dict1, and changes made in dict1 will automatically also be made in dict2.
- There are ways to make a copy, one way is to use the built-in Dictionary method copy().

Example

Make a copy of a dictionary with the copy() method:

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
mydict = dict1.copy()
print(mydict)
```

Another way to make a copy is to use the built-in function dict().

Example: Make a copy of a dictionary with the dict() function:

```
dict1 = {   "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
mydict = dict(dict1)
print(mydict)
```

Nested Dictionaries

A dictionary can contain dictionaries, this is called nested dictionaries.

Example : Create a dictionary that contain three dictionaries:

```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  } }
```

Or, if you want to add three dictionaries into a new dictionary:

Example

Create three dictionaries, then create one dictionary that will contain the other three dictionaries:

```
child1 = {   "name" : "Emil",
  "year" : 2004
}
child2 = {
  "name" : "Tobias",
  "year" : 2007
}
child3 = {
  "name" : "Linus",
  "year" : 2011
}
myfamily = {
  "child1" : child1,
  "child2" : child2,
  "child3" : child3
}
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value

update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary